

Codex Best Practices for Beginners

Build what you need without burning through your API budget

1. Introduction

Codex can build real applications quickly, but only if you guide it well. This guide covers the habits that get you working code without wasting your API budget on vague prompts, unnecessary context, or unreviewed mistakes. Every practice below exists to help you reach your goal in fewer, cheaper, more accurate steps.

2. Review Every Suggestion Before You Trust It

Codex sounds confident even when it is wrong, and fluent code is not the same as correct code. Read and understand what it generates before building on top of it, and if you cannot explain what a piece of code does, do not accept it yet.

3. Scope Your Task in specs.md Before Prompting

A vague request forces Codex to guess, producing large, unfocused code that costs more tokens to generate and is harder to check for mistakes. Before prompting, write the goal, the file(s) involved, and what “done” looks like in specs.md, then prompt against that single scoped entry.

Avoid: *“Create an end-to-end ML workflow for this dataset.”*

Prefer: *specs.md task: “Load and clean the sales CSV. Output: a function `clean_data()` in `prep.py` that removes null rows and duplicate entries.”*

4. Choose the Right Model for the Task

Not every task needs your most powerful, most expensive model, and heavier models cost more per token. Use a lightweight model for boilerplate, small fixes, and explanations, and save the advanced model for architecture decisions and hard debugging.

5. Limit Context to What the Task Needs

Every extra file you attach is billed as input tokens, and a bloated context often produces a less focused answer. Share only the file, function, or error message relevant to the current task, never the whole project “just in case.”

6. Build Incrementally

A single large request is harder to verify, and if something is wrong, it is expensive to redo the entire thing. Build one small piece at a time, such as the data layer, then the logic, then the interface, splitting any feature into two to four smaller prompts and checking each before moving on.

7. Request Targeted Changes, Not Rewrites

Rewriting a whole file regenerates code that already worked, which wastes tokens and risks introducing new bugs into working logic. Ask for a diff or a specific function update, such as updating only the validation check in `login()`, rather than asking Codex to rewrite the entire module.

8. Test Before You Trust It

Code that reads well is not the same as code that works, and building further on an untested function multiplies the cost of the mistake. Run it yourself, write a quick test for anything non-trivial, and if Codex is stuck retrying a fix, step in after one or two attempts instead of letting it loop unsupervised.

9. Manage Agent Behavior and Token Usage

Unbounded retries, stale conversations, and a cluttered workspace all quietly re-bill context you no longer need. Prefer local tools for anything deterministic, start a new conversation once context grows stale, exclude folders like `node_modules` and `dist` from your workspace, and keep `specs.md` itself lean by trimming resolved entries.

10. Quick Reference Checklist

- The task is scoped in specs.md before prompting.
- Only one task is being requested.
- The right-sized model is selected.
- Only relevant files and context are included.
- The requested change is targeted, not a full rewrite.
- A plan exists to review and test the result before building further.

11. Common Mistakes to Avoid

- Using the most capable model for every request, regardless of task size.
- Prompting Codex directly instead of scoping the task in specs.md first.
- Sending an entire repository for a minor, localized change.
- Requesting an entire application or feature in a single prompt.
- Accepting generated code without reading or testing it.
- Asking for a full rewrite instead of a targeted edit.
- Letting Codex retry a failing fix indefinitely without stepping in.
- Continuing a long conversation after context has grown stale or unfocused.